

PRÁCTICA FINAL



UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE CIENCIAS MATEMÁTICAS
MÁSTER EN INGENIERÍA MATEMÁTICA

ASIGNATURA: Base de Datos

PROFESOR: Antonio Jose Gallo Sánchez

ALUMNAS: Siria Catherine Íñiguez Brito,
Ana Marta Oliveira dos Santos

CURSO: 2025-2026

Madrid, Febrero de 2026

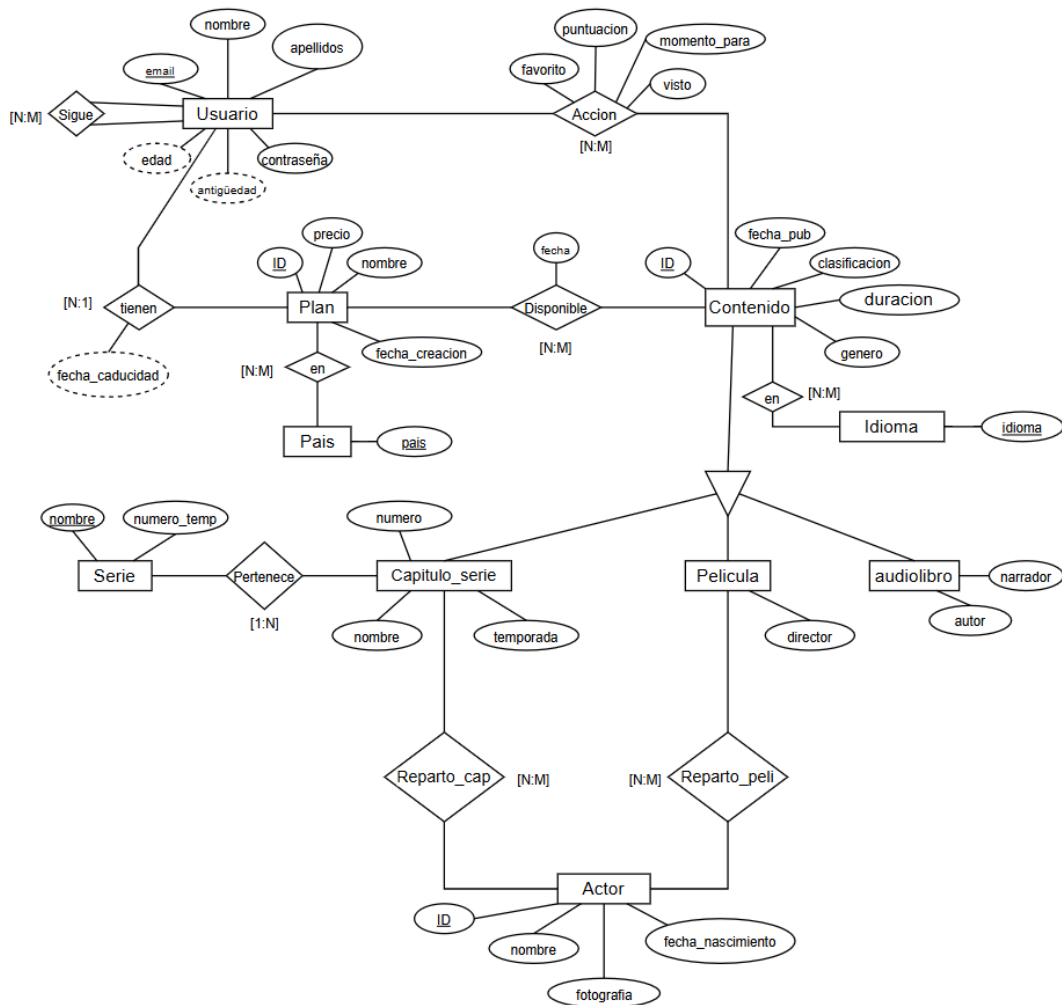
Índice

1. Parte 1: Diseño e Implementación	2
1.1. Ejercicio 1: Diseño preliminar	2
1.1.1. Modelo Entidad-Relación	2
1.1.2. Modelo relacional	3
1.1.3. Justificación y Cumplimiento de la Normalización	4
1.1.4. Decisiones no triviales y Proceso de Razonamiento	4
1.2. Ejercicio 2: Diseño adaptado a necesidades de negocio	5
1.2.1. Nuevo Modelo Relacional	5
1.2.2. Justificación de Implementación y Soluciones Alternativas	6
1.2.3. Implementación Técnica de Accesos y Seguridad	7
1.3. Ejercicio 3: Implementación del diseño	9
2. Parte 2: Consultas	10
2.1. Ejercicio 4: Consultas	10
3. Parte 3: Benchmarking	18
3.1. Ejercicio 5: Table Population	18
3.1.1. Metodología y Tecnologías	18
3.1.2. Escenarios de Volumen de Datos	18
3.1.3. Decisiones de Diseño y Reglas de Negocio	19
3.1.4. Estrategias de Optimización de Carga	19
3.1.5. Tamaño de la Base de Datos Generada	19
3.2. Ejercicio 6: Benchmarking	20
3.2.1. Métricas de Evaluación	20
3.2.2. Análisis de Resultados del Benchmarking	21
3.2.3. Optimización y Benchmarking de Consultas Especiales	23

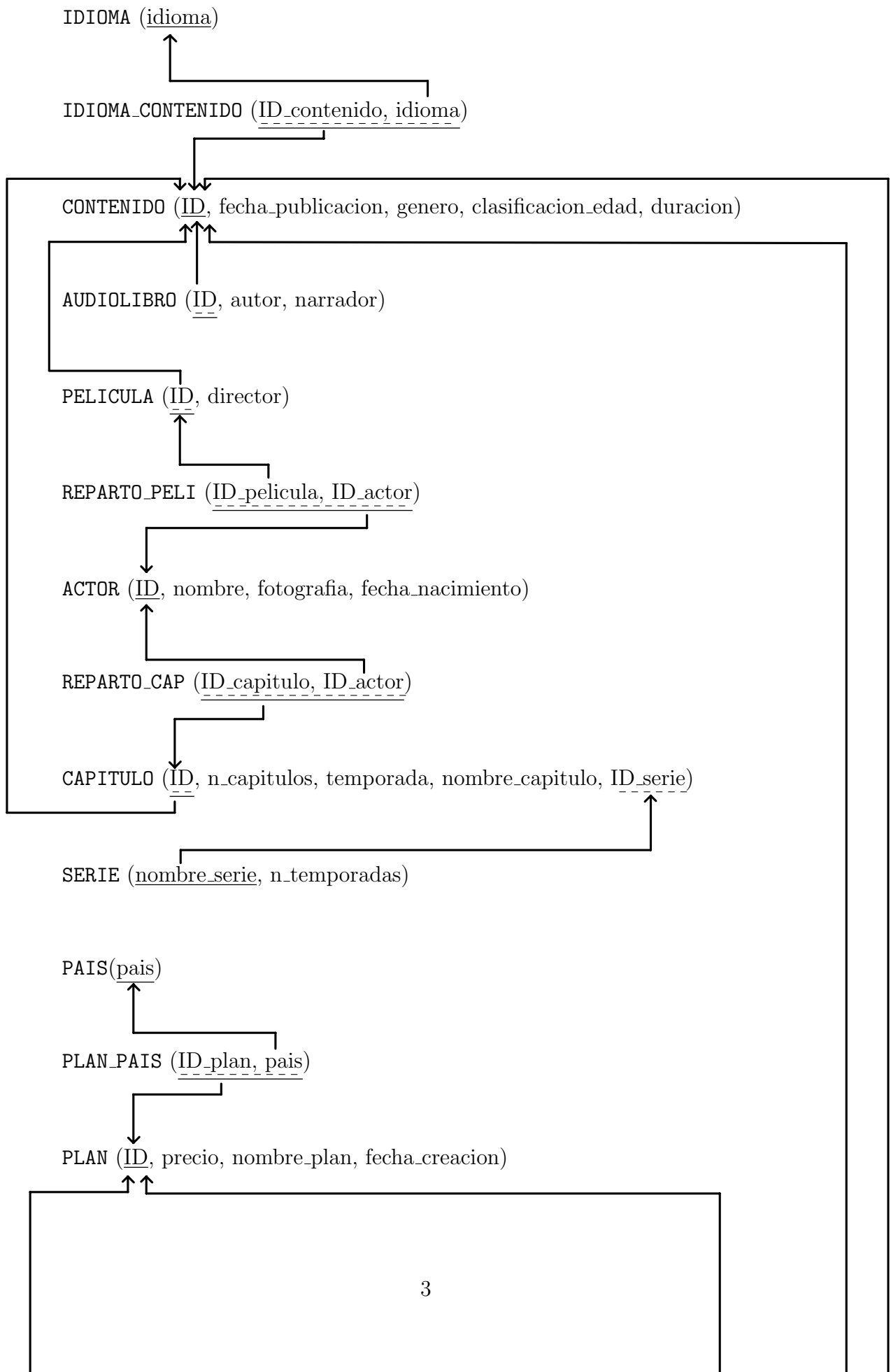
1. Parte 1: Diseño e Implementación

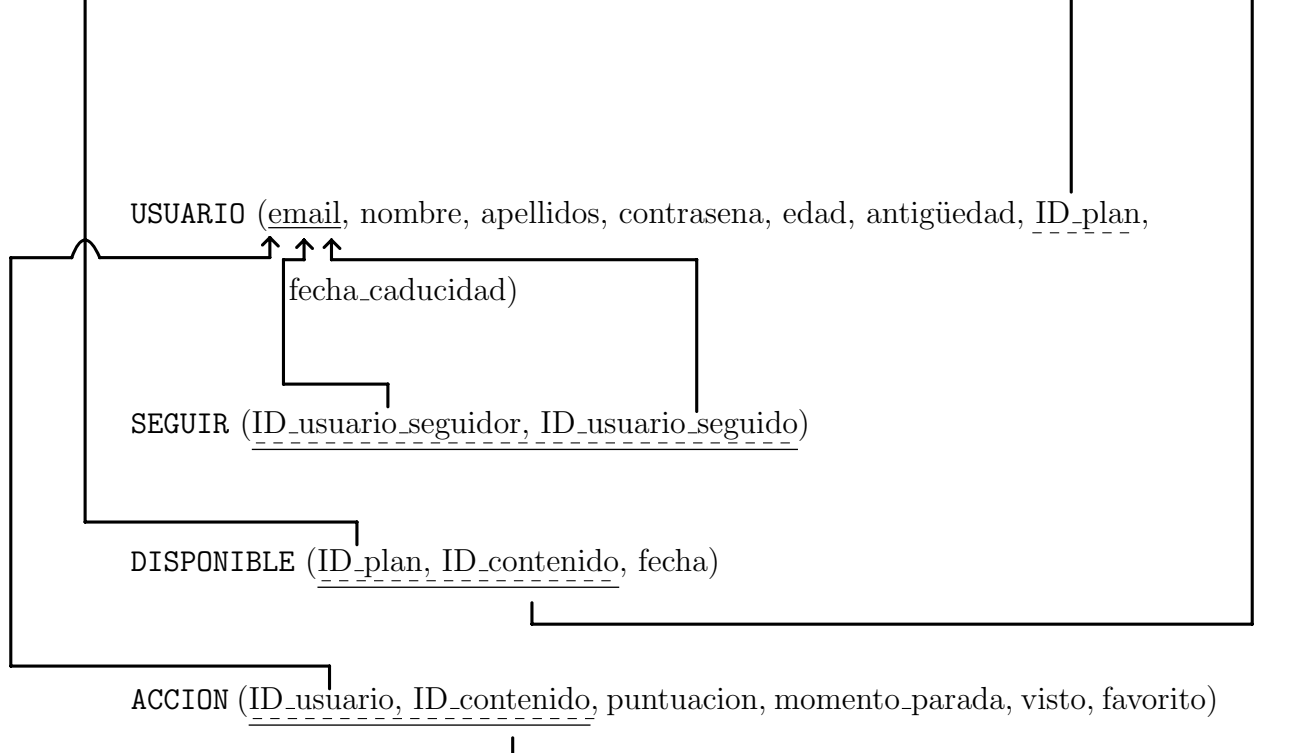
1.1. Ejercicio 1: Diseño preliminar

1.1.1. Modelo Entidad-Relación



1.1.2. Modelo relacional





1.1.3. Justificación y Cumplimiento de la Normalización

Se ha verificado que el modelo relacional propuesto cumple con la tercera forma normal (3NF) mediante el cumplimiento sucesivo de los siguientes criterios:

- **Primera forma normal (1NF):** todos los atributos contienen valores atómicos y no existen grupos repetitivos. Esto se observa en la creación de tablas independientes para entidades con múltiples valores, como por ejemplo **IDIOMA_CONTENIDO**, evitando atributos compuestos en las tablas principales.
- **Segunda forma normal (2NF):** el modelo cumple la 1NF y se ha garantizado que todos los atributos que no forman parte de la clave primaria tienen una dependencia funcional completa respecto a dicha clave, es decir, no hay dependencias parciales de la clave primaria.
- **Tercera forma normal (3NF):** el modelo cumple la 2NF y se ha eliminado cualquier dependencia transitiva. Cada atributo no clave depende únicamente de la clave primaria y no de otros atributos no clave.

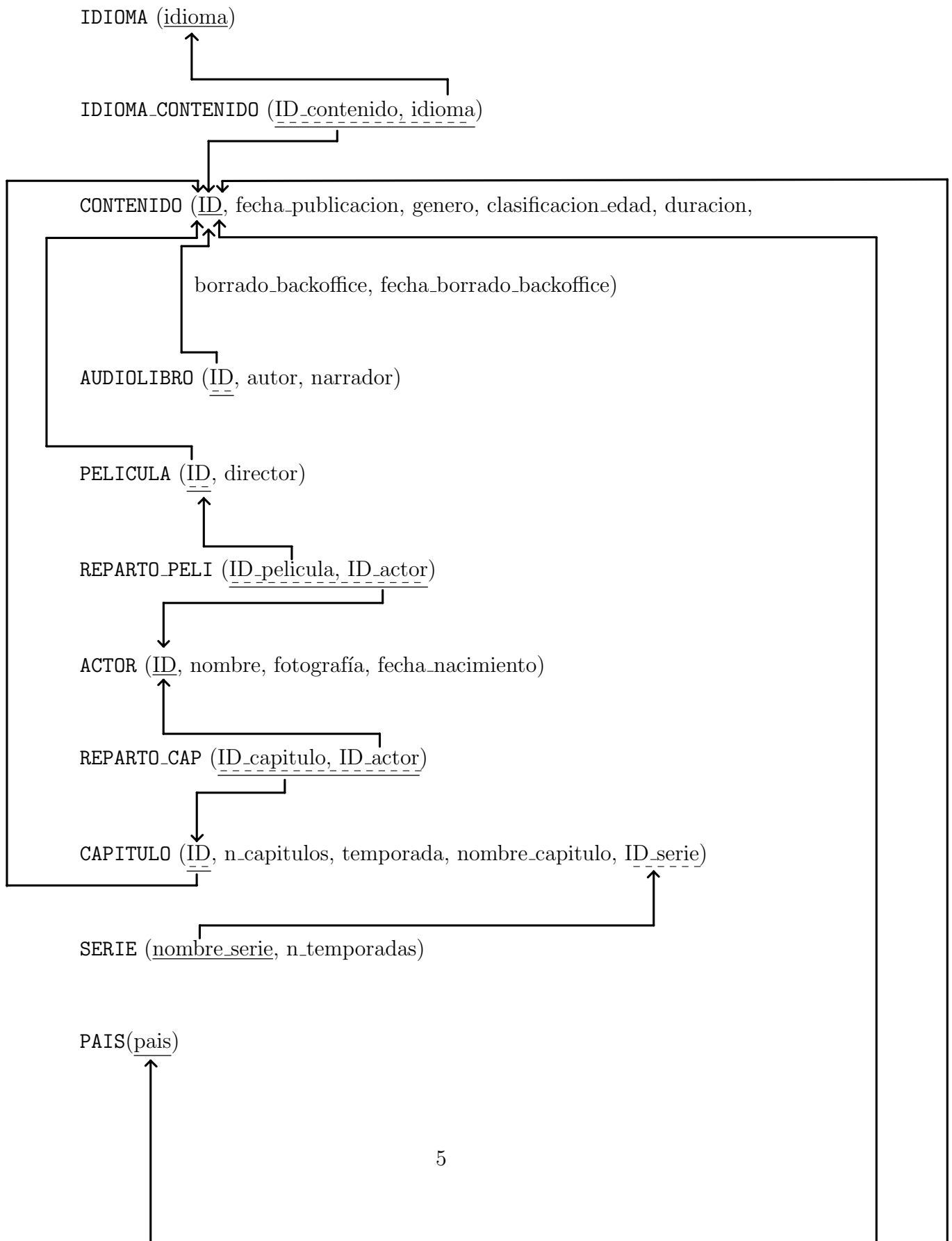
1.1.4. Decisiones no triviales y Proceso de Razonamiento

- **Traducción de la herencia de contenidos:** para mapear la superclase Contenido y sus subtipos, se ha optado por la estrategia de tabla por subclase. En el modelo relacional, las tablas hijas tienen como clave primaria el *id_contenido*, que a su vez es una clave foránea referenciando a la tabla padre. Se ha elegido esta estructura en lugar de crear una única tabla gigante con atributos nulos, ya que mantiene la base de datos más limpia y respeta la tercera forma normal, además de permitir relacionar a los actores únicamente con películas y capítulos.
- **Relación de seguimiento:** la relación de un usuario siguiendo a otro se ha modelado con una relación reflexiva N:M en el modelo E-R. En el modelo relacional, esto se traduce en la tabla intermedia **SEGUIR**, cuyas columnas actúan como clave primaria compuesta y como claves foráneas que referencian de vuelta a la tabla **USUARIO**.

1.2. Ejercicio 2: Diseño adaptado a necesidades de negocio

1.2.1. Nuevo Modelo Relacional

Se ha modificado el esquema original para incluir la partición de la entidad usuario en usuario y usuario_credencial, además de añadir los campos de control de borrado en contenido. Por fin, se añade la tabla auditoria_administracion_plan



- **Verificación de roles en triggers:** La función `CURRENT_ROLE()` presenta comportamientos inconsistentes dentro de los triggers en MySQL, ya que a menudo devuelve el rol del definidor y no del invocador.
 - **Estrategia:** para la auditoría y seguridad, se ha optado por consultar directamente la tabla del sistema `mysql.role_edges` cruzándola con `USER()` para verificar la posesión del rol.
- **Restricción selectiva de privilegios globales:** MySQL no permite revocar permisos de una entidad específica si se han concedido de forma global (`GRANT ALL ON streaming.*`).
 - **Estrategia:** Se ha aplicado el principio de mínimo privilegio, realizando `GRANT` detallados tabla por tabla para los usuarios de backup y el rol administrador.

1.2.3. Implementación Técnica de Accesos y Seguridad

Para dar cumplimiento a los requerimientos de seguridad y segregación de funciones, se detalla a continuación la configuración técnica implementada. Se detalla toda la implementación realizada abajo. Sin embargo, los códigos completos utilizados se pueden encontrar en el fichero `ejercicio2_parte1.sql`.

1. Relación de Roles, Usuarios y Privilegios

Se ha diseñado una matriz de permisos que garantiza el principio de mínimo privilegio y la protección de datos estratégicos.

Usuario / Rol	Credenciales	Permisos Asignados
backup_gen	<i>pass_segura1</i>	SELECT en tablas maestras. Sin acceso a credenciales ni interacciones (<i>accion</i>).
backup_sens	<i>pass_segura2</i>	SELECT exclusivo sobre la tabla <i>accion</i> (datos estrategicos).
web_client(Rol: backoffice)	<i>pass_segura3</i>	SELECT sobre <i>contenido_backoffice</i> y EXECUTE en procedimiento de borrado.
admin_andres (Rol: admin)	<i>SUCONTRASENA</i>	Acceso total al esquema, excepto a la tabla <i>auditoria_administrator_plan</i> .

Cuadro 1: Matriz de accesos y responsabilidades segun perfil de usuario.

2. Vistas y Seguridad de Credenciales

Se ha procedido a la segregación física de las contraseñas y a la creación de la interfaz de gestión para el backoffice.

```
CREATE TABLE usuario_credencial (
    email VARCHAR(100) PRIMARY KEY,
    password_hash CHAR(32),
    FOREIGN KEY (email) REFERENCES usuario(email) ON DELETE CASCADE
);

CREATE OR REPLACE VIEW contenido_backoffice AS
SELECT id, fecha_publicacion, clasificacion_edad, duracion, genero
FROM contenido
WHERE borrado_backoffice = 0;
```

3. Triggers de Control y Auditoría

Se han desarrollado mecanismos automáticos para gestionar el borrado lógico y la trazabilidad de cambios en los planes.

```
-- Procedimiento para ejecucion de borrado logico
DELIMITER //
CREATE PROCEDURE p_borrar_contenido_backoffice(IN p_id INT)
BEGIN
    UPDATE contenido SET borrado_backoffice = 1,
        fecha_borrado_backoffice = NOW()
    WHERE id = p_id;
END //

-- Trigger de Auditoria para Administradores
CREATE TRIGGER trigger_auditoria_plan_update
AFTER UPDATE ON plan FOR EACH ROW
BEGIN
    DECLARE es_admin INT DEFAULT 0;
    SELECT COUNT(*) INTO es_admin FROM mysql.role_edges
    WHERE FROM_USER = 'administrador' AND TO_USER = SUBSTRING_INDEX
        (USER(), '@', 1);

    IF (es_admin > 0) THEN
        INSERT INTO auditoria_administrator_plan (plan_id,
            tipo_operacion, usuario_cambio)
        VALUES (OLD.id, 'UPDATE', USER());
    END IF;
END //
DELIMITER ;
```

Nota sobre la completitud de la implementación técnica

Con el objetivo de priorizar la claridad y síntesis de este documento, se ha optado por omitir la repetición de bloques de código cuya lógica es redundante, sin que esto afecte al cumplimiento de los requisitos exigidos. Se hace constar que:

- **Auditoría Completa:** Se han implementado e integrado de forma análoga los triggers de INSERT y DELETE para la tabla `auditoria_administrator_plan`, garantizando la trazabilidad total de las operaciones realizadas por los administradores.
- **Gestión Detallada de Privilegios:** La concesión de permisos para los usuarios de backup y el rol de administrador se ha realizado mediante sentencias

`GRANT` individualizadas para cada tabla del esquema. Esta estrategia asegura el cumplimiento del principio de mínimo privilegio y sortea las limitaciones de MySQL respecto a la revocación de permisos globales.

- **Disponibilidad del Código Íntegro:** El desarrollo completo de todos los triggers, procedimientos almacenados y la configuración total de usuarios y roles se encuentra documentado y disponible para su ejecución en el archivo de script `streaming.sql` y `ejercicio2_parte1.sql` adjunto a esta práctica.

1.3. Ejercicio 3: Implementación del diseño

La implementación se ha realizado de forma local. Por tanto, se entrega una exportación autocontenida en el archivo llamado `streaming.sql`.

2. Parte 2: Consultas

2.1. Ejercicio 4: Consultas

Consulta 1

Obtener los 10 contenidos más vistos, medido como el número de usuarios que los han marcado como vistos.

```
SELECT accion.id_contenido, COUNT(*) AS vistos_totales
FROM accion
WHERE accion.visto = TRUE
GROUP BY accion.id_contenido
ORDER BY vistos_totales DESC
LIMIT 10;
```

Consulta 2

Obtener los 10 contenidos con mayor puntuación media, considerando solo aquellos que tengan al menos 50 puntuaciones.

```
SELECT
    accion.id_contenido,
    AVG(accion.puntuacion) AS puntuacion_media
FROM accion
WHERE accion.puntuacion IS NOT NULL
GROUP BY accion.id_contenido HAVING COUNT(accion.puntuacion) >= 50
ORDER BY puntuacion_media DESC
LIMIT 10;
```

Consulta 3

Encontrar los contenidos con mayor desviación en las puntuaciones (contenidos más polarizantes), considerando solo aquellos con al menos 50 votos.

```
SELECT
    id_contenido,
    STDDEV_POP(puntuacion) AS desviacion,
    COUNT(puntuacion) AS n_votos
FROM accion
WHERE puntuacion IS NOT NULL
GROUP BY id_contenido
HAVING n_votos >= 50
ORDER BY desviacion DESC
LIMIT 10;
```

Consulta 4

Obtener todos los usuarios que tienen un plan activo actualmente.

```
/*SUPONGAMOS QUE LA FECHA DE ADQUISICION ES LA MAS ACTUAL Y TODOS
LOS PLANEAS TIENEN VALIDEZ DE UN MES (30 DIAS) */
SELECT usuario.email, usuario.nombre
FROM usuario
WHERE DATE_ADD(fecha_adquisicion, INTERVAL 30 DAY) >=
    (SELECT MAX(fecha_adquisicion) FROM usuario);
```

Consulta 5

Encontrar los usuarios que han marcado más de 100 contenidos como favoritos.

```
SELECT usuario.email, usuario.nombre, COUNT(accion.favorito) AS
    n_favoritos FROM
accion JOIN usuario ON accion.id_usuario = usuario.email
GROUP BY accion.id_usuario HAVING n_favoritos > 100;

SELECT usuario.email, usuario.nombre FROM
usuario JOIN
    (SELECT accion.id_usuario, COUNT(*) AS n_favoritos
    FROM accion
    WHERE accion.favorito = TRUE
    GROUP BY accion.id_usuario HAVING n_favoritos > 100)
    AS top_favoritos
    ON top_favoritos.id_usuario = usuario.email;
```

Consulta 6

Calcular la puntuación media de los contenidos por género, considerando solo puntuaciones existentes.

```
SELECT
    contenido.genero,
    AVG(accion.puntuacion) AS media_genero
FROM contenido
JOIN accion ON contenido.id = accion.id_contenido
WHERE accion.puntuacion IS NOT NULL
GROUP BY contenido.genero;
```

Consulta 7

Obtener los 20 usuarios que han visto más contenidos en la plataforma.

```
-- Justificacion: Suponiendo que visto se considera si lo vio por
completo
SELECT
    usuario.nombre,
    accion.id_usuario,
    COUNT(*) AS n_vistos
FROM accion
JOIN usuario ON accion.id_usuario = usuario.email
WHERE accion.visto = TRUE
GROUP BY accion.id_usuario, usuario.nombre
ORDER BY n_vistos DESC
LIMIT 20;
```

Consulta 8

Recomendar contenidos a un usuario basándose en los contenidos favoritos de los usuarios a los que sigue, excluyendo los contenidos que ese usuario ya haya visto.

```
-- email de un usuario para la BBDD de 6 meses
SET @mi_email = 'user10074_kcespedes@stream.com';

-- email de un usuario para la BBDD de 1 año
SET @mi_email = 'user10008_lara76@stream.com';

SELECT
    accion.id_contenido,
    COUNT(*) AS relevancia
FROM accion
JOIN seguir ON accion.id_usuario = seguir.id_usuario_seguido
WHERE accion.favorito = TRUE
    AND seguir.id_usuario_seguidor = @mi_email
    AND accion.id_contenido NOT IN
        (SELECT id_contenido
         FROM accion
         WHERE id_usuario = @mi_email AND visto = TRUE)
GROUP BY accion.id_contenido
ORDER BY relevancia DESC
LIMIT 20;
```

Consulta 9

Encontrar los usuarios con más contenidos favoritos en común con un usuario dado.

```
-- email de un usuario para la BBDD de 6 meses
SET @email_9 = 'user1_wilfredoperez@stream.com';

-- email de un usuario para la BBDD de 1 año
SET @email_9 = 'user10008_lara76@stream.com';

SELECT a2.id_usuario AS usuario2, COUNT(*) AS favoritos_comunes
FROM accion AS a1
JOIN accion AS a2 ON a1.id_contenido = a2.id_contenido
WHERE a1.id_usuario = @email_9
    AND a1.favorito = TRUE
    AND a1.id_usuario != a2.id_usuario
    AND a2.favorito = TRUE
GROUP BY a2.id_usuario
ORDER BY favoritos_comunes DESC;
```

Consulta 10

Encontrar los usuarios que han visto todos los contenidos disponibles en un plan determinado.

```
SET @id_plan = 2;
SELECT accion.id_usuario
FROM accion
JOIN disponible ON accion.id_contenido = disponible.id_contenido
WHERE disponible.id_plan = @id_plan AND accion.visto = 1
GROUP BY accion.id_usuario
HAVING COUNT(DISTINCT accion.id_contenido) =
      (SELECT COUNT(id_contenido)
       FROM disponible
       WHERE id_plan = @id_plan);
```

Consulta 11

Obtener los contenidos que nunca han sido vistos por ningún usuario.

```
SELECT contenido.id
FROM contenido
WHERE NOT EXISTS
      (SELECT 1
       FROM accion
       WHERE accion.id_contenido = contenido.id
       AND accion.visto = 1);
```

Consulta 12

Obtener los contenidos que han sido vistos por todos los usuarios de la plataforma.

```
SELECT accion.id_contenido
FROM accion
WHERE accion.visto = 1
GROUP BY id_contenido
      HAVING COUNT(DISTINCT accion.id_usuario) =
            (SELECT COUNT(*) FROM usuario);
```

Consulta 13

Encontrar los usuarios que nunca han puntuado ningún contenido.

```
SELECT usuario.email, usuario.nombre
FROM usuario
LEFT JOIN accion ON usuario.email = accion.id_usuario
      AND accion.puntuacion IS NOT NULL
WHERE accion.id_usuario IS NULL;
```

Consulta 14

Encontrar los contenidos más abandonados, es decir, aquellos que los usuarios han empezado a ver pero no han terminado.

```
SELECT id_contenido, COUNT(id_usuario) as veces_abandonado
FROM accion
WHERE momento_parada IS NOT NULL AND visto = 0
GROUP BY id_contenido
ORDER BY veces_abandonado DESC;
```

Consulta 15

Calcular el ratio de finalización de cada contenido, definido como el número de visualizaciones completas dividido entre el número de visualizaciones iniciadas.

```
SELECT id_contenido ,
       SUM(CASE WHEN visto = 1 THEN 1 ELSE 0 END)
       / NULLIF(SUM(CASE WHEN visto = 1 OR momento_parada IS NOT
                     NULL THEN 1 ELSE 0 END), 0) as ratio_finalizacion
FROM accion
GROUP BY id_contenido;
```

Consulta 16

Para cada usuario, encontrar el género de contenido que más ha visto.

```
-- Justificacion: Usamos una CTE para rankear los generos por
-- usuario.
WITH GeneroUsuario AS (
  SELECT a.id_usuario, c.genero, COUNT(*) as total,
         RANK() OVER(PARTITION BY a.id_usuario
                     ORDER BY COUNT(*) DESC) as rnk
  FROM accion a
  JOIN contenido c ON a.id_contenido = c.id
  WHERE a.visto = 1
  GROUP BY a.id_usuario, c.genero)
SELECT id_usuario, genero FROM GeneroUsuario WHERE rnk = 1;
```

Consulta 17

Encontrar los usuarios que solo han visto contenidos de un único género.

```
SELECT a.id_usuario
FROM accion a
JOIN contenido c ON a.id_contenido = c.id
WHERE a.visto = 1
GROUP BY a.id_usuario
HAVING COUNT(DISTINCT c.genero) = 1;
```

Consulta 18

Obtener los contenidos cuya puntuación media es superior a la media global de puntuaciones de la plataforma.

```
SELECT a.id_contenido, AVG(a.puntuacion) as media_contenido
FROM accion a
CROSS JOIN (SELECT AVG(puntuacion) as global_avg
            FROM accion
            WHERE puntuacion IS NOT NULL) as m
WHERE a.puntuacion IS NOT NULL
GROUP BY a.id_contenido
HAVING media_contenido > MAX(m.global_avg);
```

En esta consulta se ha optado por utilizar una estructura CROSS JOIN frente a una CTE para mejorar el rendimiento con grandes volúmenes de datos. Al hacerlo así, MySQL calcula la media global una sola vez al principio y la deja pegada a cada fila como si fuera un dato más de la tabla. Esto evita que el motor de la base

de datos tenga que salir a buscar ese valor externo repetidamente mientras agrupa los resultados, logrando que la comparación final sea mucho más directa, sencilla y rápida para el servidor.

Consulta 19

Encontrar los actores cuyas películas han sido vistas por el mayor número de usuarios distintos.

```
SELECT r.id_actor, act.nombre, COUNT(DISTINCT a.id_usuario) as
    espectadores
FROM reparto_peli r
JOIN accion a ON r.id_pelicula = a.id_contenido
JOIN actor act ON r.id_actor = act.id
WHERE a.visto = 1
GROUP BY r.id_actor, act.nombre
ORDER BY espectadores DESC;
```

Consulta 20

Encontrar las series que han sido vistas por el mayor número de usuarios, considerando que un usuario ha visto la serie si ha visto al menos uno de sus capítulos.

```
SELECT c.id_serie, COUNT(DISTINCT a.id_usuario) as
    espectadores_serie
FROM capitulo c
JOIN accion a ON c.id = a.id_contenido
where a.visto = 1
GROUP BY c.id_serie
ORDER BY espectadores_serie DESC;
```

Consulta 21

Encontrar los pares de usuarios (seguidor-seguido) que comparten al menos 20 contenidos favoritos en común.

```
SELECT s.id_usuario_seguidor, s.id_usuario_seguido, COUNT(*) as
    favoritos_comunes
FROM seguir s
INNER JOIN accion a1 ON s.id_usuario_seguidor = a1.id_usuario
INNER JOIN accion a2 ON s.id_usuario_seguido = a2.id_usuario
WHERE a1.favorito = TRUE AND a2.favorito = TRUE AND a1.id_contenido
    = a2.id_contenido
GROUP BY s.id_usuario_seguidor, s.id_usuario_seguido
HAVING favoritos_comunes >= 20;
```

Consulta 22

Recomendar contenidos a un usuario basándose en su género más visto, excluyendo los contenidos que ya haya visto.

Para la implementación de esta consulta se han tomado las siguientes decisiones de diseño basadas en los requisitos de negocio:

- **Definición de Interés (Género Favorito):** Se ha determinado que el género más visto no debe limitarse únicamente a contenidos finalizados (`visto = 1`).

Se incluyen también los contenidos con un `momento_parada` registrado, ya que representan una intención de consumo activa por parte del usuario.

- **Criterio de Exclusión:** Siguiendo el enunciado, se excluyen de la recomendación todos los contenidos que el usuario ya haya visualizado total o parcialmente. No se excluyen aquellos marcados solo como ‘favoritos’ o ‘puntuados’ si no presentan registros de tiempo de visualización, permitiendo que el sistema sugiera elementos de su propia lista de deseos que aún no ha consumido.
- **Ranking de Calidad:** En lugar de una selección aleatoria, los resultados se ordenan por la puntuación media global de la plataforma (`AVG(puntuacion)`), asegurando que las 20 recomendaciones sean los contenidos de mayor éxito dentro del género predilecto del usuario.

```
-- email de un usuario para la BBDD de 6 meses
SELECT email INTO @id_usuario_dado FROM usuario WHERE email =
'user0_32831c@stream.com' LIMIT 1;

-- email de un usuario para la BBDD de 1 año
SELECT email INTO @id_usuario_dado FROM usuario WHERE email =
'user10008_lara76@stream.com'; LIMIT 1;

-- Identificamos el genero con mas contenidos finalizados (visto =
1)
WITH GeneroMasVisto AS (
SELECT c.genero
FROM accion a
JOIN contenido c ON a.id_contenido = c.id
WHERE a.id_usuario = @id_usuario_dado
AND (a.visto = 1 OR a.momento_parada IS NOT NULL)
GROUP BY c.genero
ORDER BY COUNT(*) DESC
LIMIT 1)

SELECT c.id, c.genero, COALESCE(AVG(a_media.puntuacion), 0) as
puntuacion_media
FROM contenido c
-- Unimos con accion solo para calcular la media de los contenidos
del genero elegido
LEFT JOIN accion a_media ON c.id = a_media.id_contenido
AND a_media.puntuacion IS NOT NULL

-- Usamos un LEFT JOIN para excluir lo que el usuario ya vio (Mas
eficiente que NOT IN)
LEFT JOIN accion a_visto ON c.id = a_visto.id_contenido
AND a_visto.id_usuario = @id_usuario_dado
WHERE c.genero = (SELECT genero FROM GeneroMasVisto)
AND (a_visto.id_contenido IS NULL
OR (a_visto.visto = 0 AND a_visto.momento_parada
IS NULL))
GROUP BY c.id, c.genero
ORDER BY puntuacion_media DESC
LIMIT 20;
```

Consulta 23

Mostrar las series con mayor número de actores distintos.

```
SELECT id_serie, COUNT(DISTINCT rc.id_actor) as num_actores
FROM capitulo cap
JOIN reparto_cap rc ON cap.id = rc.id_capitulo
GROUP BY id_serie
ORDER BY num_actores DESC;
```

Consulta 24

Mostrar el actor que ha trabajado con más actores distintos.

```
WITH ActorContenido AS (
    SELECT id_actor, id_pelicula as id_cont FROM reparto_peli
    UNION
    SELECT id_actor, id_capitulo as id_cont FROM reparto_cap)

SELECT ac1.id_actor, COUNT(DISTINCT ac2.id_actor) - 1
    as companeros
FROM ActorContenido ac1
JOIN ActorContenido ac2 ON ac1.id_cont = ac2.id_cont
GROUP BY ac1.id_actor
ORDER BY companeros DESC LIMIT 1;
```

Consulta 25

Mostrar los actores que han participado en más contenidos que cualquier otro actor de su misma edad.

```
WITH ContenidosPorActor AS (
    SELECT a.id, a.nombre,
        TIMESTAMPDIFF(YEAR, a.fecha_nacimiento, CURDATE()) AS
        edad,
        (COUNT(DISTINCT rp2.id_pelicula) + COUNT(DISTINCT rc2.
            id_capitulo)) AS total_contenidos
    FROM actor a
    LEFT JOIN reparto_peli rp2 ON a.id = rp2.id_actor
    LEFT JOIN reparto_cap rc2 ON a.id = rc2.id_actor
    GROUP BY a.id, a.nombre, a.fecha_nacimiento),

MaxContenidosPorEdad AS (
    SELECT edad, MAX(total_contenidos) as max_contenidos
    FROM ContenidosPorActor
    GROUP BY edad)

SELECT cpa.id, cpa.nombre, cpa.edad, cpa.total_contenidos
FROM ContenidosPorActor cpa
JOIN MaxContenidosPorEdad mce ON cpa.edad = mce.edad AND cpa.
    total_contenidos = mce.max_contenidos;
```

3. Parte 3: Benchmarking

3.1. Ejercicio 5: Table Population

En esta sección se detalla el proceso de generación de datos sintéticos diseñado para simular la actividad de la plataforma en dos escenarios temporales distintos. La automatización se ha realizado mediante un script desarrollado en Python (`ejercicio5.py`), garantizando la coherencia de los datos y el cumplimiento de las reglas de negocio.

3.1.1. Metodología y Tecnologías

Para la generación de la población se han utilizado las siguientes herramientas técnicas:

- **Python 3.13:** Como lenguaje principal para la lógica de generación.
- **Librería Faker:** Utilizada para generar datos realistas como nombres de usuarios, correos electrónicos y fechas de nacimiento, configurada con el perfil `es_ES` para mantener el contexto local.
- **Lógica Probabilística:** Implementada mediante la librería `random` para cumplir con las distribuciones de uso (porcentajes de planes activos, contenidos vistos, etc.) especificadas en el enunciado.
- **Pre-procesamiento de Seguridad:** El hacheado de contraseñas mediante MD5 se realiza en Python antes de la inserción, optimizando el rendimiento del servidor.
- **Reproducibilidad:** Se ha fijado una semilla aleatoria (2026) para asegurar que ambos escenarios de datos sean consistentes y comparables en las pruebas de rendimiento.

3.1.2. Escenarios de Volumen de Datos

Se han definido dos configuraciones principales que dictan el tamaño de la base de datos resultante:

Entidad	Escenario A (6 meses)	Escenario B (1 año)
Usuarios	25 000	40 000
Películas	3 500	5 500
Audiolibros	1 200	2 000
Series	300	500
Actores	4 000	6 000

Cuadro 2: Volumen de datos por escenario según las especificaciones del proyecto.

3.1.3. Decisiones de Diseño y Reglas de Negocio

El script `ejercicio5.py` aplica de forma automática las siguientes restricciones y lógicas de población:

- **Usuarios:** Cada usuario tiene una edad entre 18 y 70 años, con una probabilidad del 78 % de poseer un plan activo.
- **Relaciones Sociales:** Se modela una relación de seguimiento (**seguir**) no recíproca de 0 a 50 usuarios por persona.
- **Jerarquía de Contenidos:** Las series se estructuran en 1-5 temporadas con 6-12 capítulos cada una.
- **Interacciones (Tabla `accion`):** Es la tabla de mayor volumetría. Cada usuario realiza entre 100 y 400 interacciones, distribuidas según los siguientes pesos probabilísticos:
 - 60 % de contenidos vistos completamente.
 - 20 % de contenidos abandonados (con registro de momento de parada).
 - 30 % de contenidos marcados como favoritos.
 - 40 % de contenidos puntuados (escala del 1 al 5).

Se ha aplicado una lógica `if/elif` para garantizar exclusividad entre vistos (60 %) y abandonados (20 %).

- **Coherencia Técnica:** El campo `momento_parada` se valida contra la duración real del contenido almacenada en un diccionario temporal (`duraciones_contenido`).

3.1.4. Estrategias de Optimización de Carga

Para gestionar la carga de millones de registros de forma eficiente, se han implementado las siguientes optimizaciones en el archivo SQL resultante:

- **Control de Restricciones:** Se deshabilitan `FOREIGN_KEY_CHECKS` y `UNIQUE_CHECKS` durante la importación para maximizar la velocidad de escritura.
- **Inserción Masiva (Bulk Insert):** Se agrupan registros en lotes de 20,000 filas por sentencia `INSERT`, reduciendo la latencia de comunicación con el motor.
- **Gestión de Transacciones:** Se realizan `COMMIT` periódicos (cada 100,000 acciones) para liberar el *Undo Log* y evitar el agotamiento de la memoria RAM del servidor.

3.1.5. Tamaño de la Base de Datos Generada

Tras la importación de los archivos `.sql`, se han registrado los siguientes tamaños totales mediante la consulta a `information_schema` (en el fichero `tamano`):

- **Escenario A (6 meses):** 1294.69 MB.
- **Escenario B (1 año):** 2071.02 MB.

3.2. Ejercicio 6: Benchmarking

El objetivo de esta sección es evaluar la escalabilidad y eficiencia del diseño de la base de datos frente al incremento progresivo en la volumetría de los datos. En un ecosistema real de *streaming*, el rendimiento de las consultas analíticas es determinante para la toma de decisiones estratégicas y la personalización de la experiencia del usuario. Por ello, este análisis se centra en identificar los límites operativos del sistema cuando la actividad de los usuarios se duplica en el horizonte temporal.

Para este estudio, se han ejecutado las consultas definidas en la Sección 2 bajo dos escenarios diferenciados:

- **Escenario A (6 meses):** Representa una plataforma en fase de crecimiento temprano y un volumen de interacciones proporcional a dicho periodo.
- **Escenario B (1 año):** Simula la consolidación de la plataforma, y un catálogo de contenidos y acciones significativamente más extenso.

A través de los resultados comparativos, se pretende dar respuesta a tres ejes fundamentales:

1. **Análisis de Escalabilidad:** Observar cómo aumenta el tiempo de ejecución a medida que crece el volumen de registros en las tablas principales.
2. **Identificación de Cuellos de Botella:** Detectar consultas con un factor de empeoramiento desproporcionado, analizando si el origen es la complejidad lógica de la sentencia SQL o la estructura física de la base de datos.
3. **Efectividad de la Optimización:** Cuantificar la mejora tras implementar índices compuestos o reescritura de código en las consultas con mayor latencia.

3.2.1. Métricas de Evaluación

Con el fin de normalizar la comparativa de rendimiento, se han definido las siguientes métricas calculadas a partir de los tiempos de ejecución:

- **Tiempos de ejecución (6 meses / 1 año):** Estas dos columnas registran el tiempo real, expresado en segundos, que emplea el motor de la base de datos para completar cada consulta bajo condiciones de carga determinadas.
- **Factor de Empeoramiento (F. Emp.):** Indica el crecimiento relativo de la latencia. Se define como:

$$\text{F. Emp.} = \frac{\text{Tiempo (1 año)}}{\text{Tiempo (6 meses)}}$$

- **Incremento Porcentual (Inc. %):** Permite visualizar rápidamente el impacto del volumen de datos en términos de latencia adicional para el sistema:

$$\text{Inc. \%} = \left(\frac{\text{Tiempo (1 año)} - \text{Tiempo (6 meses)}}{\text{Tiempo (6 meses)}} \right) \times 100$$

#	Nombre de la Consulta	6 meses(s)	1 año (s)	F. Emp.	Inc. %
1	Top 10 contenidos más vistos	820.094	1553.672	1.895	89.450
2	Top 10 mejor puntuados(min 50)	817.140	1505.922	1.843	84.292
3	Contenidos más polarizantes	817.891	1502.797	1.837	83.740
4	Usuarios con plan activo	0.047	0.078	1.660	65.957
5	Usuarios con >100 favoritos	27.36	46.750	1.709	70.870
6	Puntuación media por género	824.172	1520.219	1.845	84.454
7	Top 20 usuarios más activos	46.047	78.812	1.712	71.156
8	Rec. por favoritos de seguidos	0.046	0.032	0.696	-30.435
9	Usuarios con gustos afines	6.781	6.094	0.899	-10.131
10	Usuarios: plan completo visto	43.453	95.782	2.204	120.427
11	Contenidos nunca vistos	1.094	16.547	15.125	1412.523
12	Contenidos vistos por todos	6.515	1504.641	230.950	22995.027
13	Usuarios que nunca puntuaron	4.422	7.250	1.640	63.953
14	Contenidos más abandonados	817.297	1507.062	1.844	84.396
15	Ratio de finalización	63.891	71.218	1.115	11.468
16	Género más visto por usuario	57.406	230.950	2.025	102.505
17	Usuarios de género único visto	14.454	25.547	1.767	76.747
18	Contenidos sobre la media global	135.015	154.078	1.141	14.119
19	Actores con mayor n° de usuarios	20.906	76.61	3.664	266.450
20	Series con más usuarios	5.609	39.421	7.028	602.817
21	Pares seguidor-seguido	562.547	936.297	1.664	66.439
22	Rec. por género más visto	925.672	1539.531	1.663	66.315
23	Series con mayor n° de actores	0.093	0.094	1.011	1.075
24	Actor con + co-actores distintos	0.328	2.656	8.091	709.146
25	Actores líderes por rango de edad	0.312	0.500	1.603	60.256

Cuadro 3: Comparativa de rendimiento entre Escenario A y Escenario B.

3.2.2. Análisis de Resultados del Benchmarking

En esta sección se analiza el impacto del volumen de datos en el rendimiento comparando el escenario A y escenario B.

Relación entre Rendimiento y Volumen de Datos

La tendencia general indica que el rendimiento es inversamente proporcional al volumen de datos: el tiempo de ejecución aumenta conforme crece el n° de registros. No obstante, este impacto no es uniforme y depende de la complejidad de la consulta:

- **Escalabilidad Lineal:** Consultas que operan sobre tablas maestras o filtros directos. Un ejemplo claro es la consulta #4, que presenta un incremento del 65.9%, cifra alineada con el crecimiento de la tabla de usuarios.

- **Escalabilidad Polinómica:** Consultas que requieren agregaciones masivas de registros de actividad (*accion*). Las consultas #1 y #14 muestran factores de empeoramiento cercanos a 1.8, lo cual es coherente con la duplicación del historial de visualizaciones.
- **Degradación Crítica:** Consultas donde el diseño algorítmico o la lógica de comparación de conjuntos provoca un crecimiento exponencial del tiempo de respuesta. Ejemplos notables son las consultas #11 y #12, donde el incremento porcentual supera el 1,400 % y 22,000 % respectivamente, evidenciando una falta de escalabilidad ante el aumento del volumen de datos.

Consultas con Empeoramiento Desproporcionado

Se han detectado casos donde el tiempo de espera (latencia) crece de forma alarmante, aumentando mucho más de lo que cabría esperar por el simple incremento natural de usuarios y datos en la plataforma::

#	Nombre de la Consulta	Factor Emp.	Inc. %
12	Contenidos vistos por todos	230.950	22995.03 %
11	Contenidos nunca vistos	15.125	1412.52 %
20	Series con más usuarios	7.028	602.82 %

Cuadro 4: Consultas con mayor degradación de rendimiento.

La disparidad en la degradación responde a la naturaleza de las operaciones lógicas de SQL:

- **Criterios de Totalidad (Consulta 12):** Para verificar qué contenidos han sido vistos por *todos* los usuarios, el motor debe realizar una comparación cruzada masiva. Al aumentar simultáneamente el catálogo y los usuarios, el número de comparaciones escala de forma multiplicativa ($O(N \times M)$), convirtiéndose en la operación más costosa.
- **Verificación de Ausencia (Consulta 11):** Identificar lo que *no* existe requiere escaneos exhaustivos para confirmar la inexistencia del ID en millones de registros de la tabla *accion*. Probar una ausencia es computacionalmente más caro que confirmar una presencia.
- **Agregaciones de Granularidad Fina (Consulta 20):** Contar usuarios únicos por serie implica navegar por la jerarquía *serie-capitulo-accion*. El filtrado de duplicados (*DISTINCT*) sobre una tabla de hechos que ha doblado su tamaño dispara el uso de memoria y CPU.

Observación sobre los Resultados Negativos (Consultas 8 y 9)

Es imperativo notar que las consultas #8 (Recomendaciones por seguidos) y #9 (Gustos afines) presentan incrementos negativos (una mejora de tiempo en el escenario más grande).

Este resultado no refleja una mayor eficiencia, sino como una consecuencia de la **independencia de los escenarios de prueba**. Al ser bases de datos generadas de forma aleatoria y distinta para cada escenario, la estructura de las relaciones (usuarios que siguen a otros o favoritos en común) no es constante. Si en el escenario B los usuarios seleccionados aleatoriamente para el test tienen menos conexiones o favoritos que los del escenario A, la consulta procesa menos ramificaciones lógicas, resultando en un tiempo menor que no es representativo de la escalabilidad real.

3.2.3. Optimización y Benchmarking de Consultas Especiales

En esta fase se detalla la estrategia técnica para optimizar las consultas con mayor carga computacional (#1, #5, #7 y #14).

Consulta 1: Contenidos más vistos

En el escenario base, el motor de la base de datos requería un tiempo excesivo de procesamiento debido al gran volumen de registros que debía leer de forma secuencial.

Escenario	Antes (V1)	Después (V2: Índice + Consulta)
6 Meses	820.094 s	65.250s (Índ.) + 3.578s (Ejec.)
1 Año	1553.672 s	32.328s (Índ.) + 6.047s (Ejec.)

Cuadro 5: Benchmark Consulta 1: Reducción drástica de latencia.

Listing 1: Version Optimizada - Consulta 1

```
/* Creacion de indice compuesto para cobertura total */
CREATE INDEX idx_visto_idcont ON accion (visto, id_contenido);

SELECT accion.id_contenido, COUNT(*) AS vistos_totales
FROM accion
JOIN contenido ON accion.id_contenido = contenido.id
WHERE accion.visto = TRUE
GROUP BY accion.id_contenido
ORDER BY vistos_totales DESC
LIMIT 10;
```

Para mitigar la latencia, se implementó un **índice compuesto** (idx_visto_idcont) diseñado bajo el principio de **cobertura total**. Al integrar el estado de visualización y el identificador del contenido en una misma estructura indexada, el motor evita el acceso redundante a las páginas de datos de la tabla física. Esta estrategia transforma una búsqueda secuencial masiva en un acceso directo, permitiendo que el escenario de un año se resuelva en apenas 6 segundos de tiempo neto de ejecución.

Consulta 5: Usuarios con más de 100 favoritos

Esta consulta identifica a los usuarios más activos basándose en la cantidad de contenidos marcados como favoritos. El desafío técnico reside en realizar el conteo sobre la tabla de acciones antes de cruzar los datos con la tabla de usuarios, evitando así procesar información innecesaria de perfiles que no cumplen el criterio.

Escenario	V1 (Original)	V2 (Subconsulta)	V3 (Índice + Sub.)
6 Meses	27.360 s	8.687 s	24.375s (Índ.) + 1.687s (Ejec.)
1 Año	46.750 s	14.610 s	39.375s (Índ.) + 2.938s (Ejec.)

Cuadro 6: Benchmark Consulta 5: Eficiencia por agregación previa y uso de índices.

Listing 2: Primer intento de optimización - Consulta 5

```
SELECT usuario.email, usuario.nombre, top_favoritos.n_favoritos
FROM usuario
JOIN (SELECT accion.id_usuario, COUNT(*) AS n_favoritos
      FROM accion
      WHERE accion.favorito = TRUE
      GROUP BY accion.id_usuario HAVING n_favoritos > 100)
      AS top_favoritos
ON top_favoritos.id_usuario = usuario.email;
```

Listing 3: Versión Optimizada - Consulta 5

```
/* Creacion de indice para acelerar el filtrado y agrupamiento */
CREATE INDEX idx_accion_favorito_usuario ON accion(favorito,
id_usuario);

/* Consulta optimizada mediante subconsulta (agregacion previa) */
SELECT u.email, u.nombre, top_favoritos.n_favoritos
FROM usuario u
INNER JOIN (SELECT id_usuario, COUNT(*) AS n_favoritos
            FROM accion
            WHERE favorito = TRUE
            GROUP BY id_usuario
            HAVING n_favoritos > 100)
            AS top_favoritos
ON top_favoritos.id_usuario = u.email;
```

La mejora en esta consulta radica en una **reestructuración lógica** que prioriza la agregación previa. Mediante una subconsulta apoyada por el índice compuesto `idx_accion_favorito_usuario`, el sistema pre-calcula los volúmenes de favoritos antes de realizar cualquier vínculo de identidad con la tabla de usuarios.

Esta estrategia impide que el motor sature la memoria intentando procesar metadatos de perfiles que finalmente no cumplirán el criterio del `HAVING >100`. El impacto es drástico en el escenario de mayor volumen (1 año), donde el tiempo de ejecución neto cayó de **46.750 segundos** a apenas **2.938 segundos**. Gracias a este aislamiento de procesos, se logra una optimización superior al **93 %** en el tiempo de respuesta, garantizando que el coste computacional crezca de forma controlada ante el aumento del volumen de actividad.

Consulta 7: Ranking de usuarios (Vistos)

Esta consulta genera un ranking de los 20 usuarios con mayor número de contenidos visualizados. En el planteamiento inicial, el motor sufría una sobrecarga al intentar cruzar cada registro de actividad con la tabla de usuarios antes de realizar el conteo, lo que generaba un cuello de botella innecesario en la memoria.

Escenario	V1 (JOIN)	V2 (Sin JOIN)	V3 (Índice + Hint)
6 Meses	46.047 s	9.140 s	26.203s (Índ.) + 5.109s (Ejec.)
1 Año	78.812 s	63.156 s	41.687s (Índ.) + 8.296s (Ejec.)

Cuadro 7: Benchmark Consulta 7: Impacto de la reducción de JOINS y uso de Hints.

Listing 4: Versión Optimizada - Consulta 7

```
-- Creacion del indice para optimizar el agrupamiento por usuario
*/
CREATE INDEX idx_usuario_visto ON accion(id_usuario, visto);

SELECT id_usuario, COUNT(id_usuario) AS n_vistos
FROM accion USE INDEX (idx_usuario_visto)
WHERE visto = 1
GROUP BY id_usuario
ORDER BY n_vistos DESC
LIMIT 20;
```

La clave de esta optimización fue simplificar la operación realizando el conteo directamente en la tabla de actividad (*accion*) y prescindiendo del *JOIN* con la tabla de usuarios. Además, se implementó un **Optimizer Hint** (*USE INDEX*) para forzar al motor a utilizar el índice compuesto *idx_usuario_visto*, garantizando que la base de datos no elija una ruta de ejecución menos eficiente.

Gracias a este enfoque, se logró que la consulta sea extremadamente ligera, ya que el motor solo necesita leer el índice para obtener los resultados. La mejora es notable en el escenario de mayor volumen (1 año), donde el tiempo de ejecución de la consulta cayó de casi 79 segundos a poco más de 8 segundos, eliminando procesos de carga de datos innecesarios y optimizando el uso del procesador.

Consulta 14: Contenidos más abandonados

Esta consulta identifica los contenidos que los usuarios han iniciado pero no han finalizado (*visto = 0* y *momento_parada* no nulo). En el escenario original, el motor de búsqueda debía realizar un escaneo completo de la tabla *accion*, lo que generaba un factor de empeoramiento de 1.844 al duplicar el volumen de datos.

Escenario	Antes (V1: Original)	Después (V2: Índice + Consulta)
6 Meses	817.297 s	20.156s (Índ.) + 2.953s (Ejec.)
1 Año	1507.062 s	34.141s (Índ.) + 4.812s (Ejec.)

Cuadro 8: Benchmark Consulta 14: Optimización mediante índice de cobertura.

La optimización de esta consulta se basó en la implementación de un **índice de cobertura multivariable** (*idx_visto_parada_cont*) que integra todos los campos necesarios para el filtrado y la agrupación. Al forzar su uso mediante la instrucción *USE INDEX*, el motor resuelve la operación leyendo exclusivamente la estructura indexada y evita el acceso a la tabla física.

Esta estrategia elimina los escaneos secuenciales y localiza los registros de forma instantánea, logrando estabilizar el rendimiento y mitigar el factor de empeoramiento inicial. Como resultado, el tiempo de ejecución en el escenario de un año se redujo drásticamente de **1507 segundos** a menos de **5 segundos**. Esta intervención representa una mejora de eficiencia superior al **99.6 %**, neutralizando el impacto del crecimiento del volumen de datos en el sistema.

Listing 5: Version Optimizada - Consulta 14

```
-- Creacion del indice compuesto para optimizar el filtro y la
    agregacion

CREATE INDEX idx_visto_parada_cont ON accion (visto, momento_parada
    , id_contenido);

-- Consulta optimizada utilizando el indice de cobertura
SELECT
    id_contenido ,
    COUNT(*) AS veces_abandonado
FROM accion USE INDEX (idx_visto_parada_cont)
WHERE visto = 0
    AND momento_parada IS NOT NULL
GROUP BY id_contenido
ORDER BY veces_abandonado DESC;
```